

Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
Campus Bragança Paulista

Andrey Nagatani BP3044505
David do Nascimento Martins BP3060616

Boost Plus

Projeto de Extensão do 2º. módulo do Curso ADS

Extensão como Metodologia de Ensino 2 (BRAEXT2)
Prof. Luiz Gustavo Diniz de Oliveira Veras

Sumário

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	Introdução	4
2.2	Relato de Contato com a Comunidade Externa	4
2.2.1	Identificação da Demanda e Público-Alvo	4
2.2.2	Ciclo de Feedback e Metodologia de Interação	5
2.3	Características da Aplicação	5
2.3.1	Atores e seus papéis	5
2.3.2	Classes de usuários e características	5
2.4	Arquitetura física	6
2.4.1	Tecnologias utilizadas	6
2.5	Arquitetura lógica	7
2.5.1	Requisitos funcionais	10
2.5.2	Requisitos de regras de negócio	11
2.5.3	Requisitos não funcionais	12
2.5.4	Especificações funcionais	13
2.5.4.1	Especificação UC 0001: Registrar Manutenção	14
2.5.4.2	Especificação UC 0002: Cadastrar Veículo	15
2.5.4.3	Especificação UC 0003: Consultar Histórico	16
2.5.4.4	Especificação UC 0004: Atualizar Odômetro	17
2.5.4.5	Especificação UC 0005: Gerar Alerta de Troca	18
2.5.4.6	Especificação UC 0006: Manter Categorias	19
2.5.5	Diagrama de Atividades	20
2.5.6	Diagrama de classes de domínio	22
2.5.7	Modelo Entidade Relacionamento (M.E.R)	23
3	CODIFICAÇÃO	25
	REFERÊNCIAS	26

1 Introdução

O projeto Boost+ nasceu da observação direta de lacunas na gestão de serviços automotivos, motivada após uma aproximação na rotina da oficina Boost Motorsport. Durante essa experiência, constatou-se que tanto prestadores de serviço da área quanto proprietários de veículos enfrentam dificuldades em manter um histórico preciso e acessível sobre as manutenções realizadas. A falta de um controle sobre a durabilidade e os prazos de validade de componentes trocados gera incertezas e dificulta as manutenções preventivas.

A plataforma surge com o propósito fundamental de atuar como um elo estratégico entre a oficina e o cliente. O objetivo principal é centralizar informações críticas de manutenção em uma interface intuitiva, permitindo que o usuário acompanhe a saúde do seu veículo e que haja uma base de dados estruturada para os registros. Ao facilitar o monitoramento das condições das peças e a quilometragem do veículo, o Boost+ busca promover uma gestão mais transparente, segura e eficiente da vida útil dos veículos.

2 Desenvolvimento

2.1 Introdução

O projeto Boost+ surgiu da observação direta de uma necessidade no setor de serviços automotivos, identificada durante visitas à oficina Boost Motorsport. Percebeu-se que, na ausência de ferramentas digitais, o controle sobre as manutenções realizadas e a validade dos componentes substituídos tornava-se uma tarefa complexa e sujeita a falhas dos proprietários de veículos.

Muitas vezes, informações vitais sobre itens do automóvel e os prazos de revisão preventiva acabam se perdendo em registros informais ou processos manuais sem o devido acompanhamento técnico (um adesivo colado no parabrisa não comporta todos os detalhes). Pensando nisso, a ideia foi criar uma plataforma que centralize essas interações, estabelecendo um canal de comunicação direto e organizado entre a oficina mecânica e o cliente.

O funcionamento do sistema é focado na inserção dos dados técnicos: o mecânico insere dados específicos como data, quilometragem, especificações de produtos e prazos de durabilidade. Por outro lado, o cliente, através de um aplicativo móvel, pode consultar esse histórico e atualizar o odômetro do veículo para que o sistema calcule automaticamente a vida útil restante de cada componente monitorado.

O principal objetivo do Boost+ é transformar a gestão de manutenção em uma experiência transparente e preventiva, garantindo a segurança do condutor e a preservação do seu veículo. Dessa forma, a plataforma busca substituir o problema dos prazos manuais por um fluxo de trabalho automatizado e confiável, beneficiando tanto a eficiência operacional da oficina quanto a tranquilidade do cliente.

2.2 Relato de Contato com a Comunidade Externa

2.2.1 Identificação da Demanda e Público-Alvo

O processo iniciou-se com uma visita técnica à oficina local, onde foi identificado que a gestão do histórico de manutenção ainda era realizada de forma fragmentada, gerando insegurança para o proprietário do veículo e retrabalho para o prestador de serviço. O público-alvo foi segmentado em duas frentes:

- Mecânicos: Profissionais que necessitam de agilidade na inserção de dados técnicos e organização de categorias.

- Clientes (Proprietários): Condutores que buscam transparência e alertas preventivos para a manutenção do seu veículo.

2.2.2 Ciclo de Feedback e Metodologia de Interação

A interação com o público-alvo ocorreu por meio de ciclos de feedback, onde os requisitos funcionais foram refinados com base na prática profissional:

- Diagnóstico Participativo: Entrevistas com os mecânicos para entender quais dados são cruciais no momento da troca de óleo e manutenções do veículo.
- Validação de Protótipos (IHC): Apresentação de protótipos da interface web (React) e mobile (Flutter) para avaliar se a usabilidade atende ao ambiente de uma oficina. Um exemplo dessa interação foi a inclusão da funcionalidade de Gerenciamento de Categorias (RF-13), que foi solicitada pelo profissional para permitir a divisão dos itens utilizados nas diferentes manutenções.
- Ajuste de Regras de Negócio: A regra de integridade do odômetro (RGN-05), que impede a inserção de quilometragem inferior à anterior, foi estabelecida após a percepção da necessidade de garantir a veracidade do histórico.

2.3 Características da Aplicação

2.3.1 Atores e seus papéis

- Mecânico: Responsável por realizar o cadastro técnico das manutenções, especificando produtos (óleo, peças), prazos e gerenciando as categorias de peças e serviços.
- Cliente: Proprietário do veículo que consulta o histórico e atualiza a quilometragem atual para que o sistema calcule a vida útil dos componentes.
- Administradores: Gerencia usuários, oficinas cadastradas e categorias de serviços.

2.3.2 Classes de usuários e características

Os usuários e suas características são descritas no Quadro 1.

Quadro 1 – Classes de usuários e características

Classe	Descrição
Administradores	Responsáveis por gerenciar o sistema, manter as contas e perfis de mecânicos e clientes, além de garantir a integridade dos dados e o bom funcionamento global da aplicação.
Mecânicos	Profissionais responsáveis pelo registro técnico das manutenções, cadastro de veículos e gerenciamento das categorias de serviços e peças, visando a organização dos registros históricos da oficina.
Clientes (Proprietários)	Principais usuários da plataforma mobile. Podem cadastrar seus veículos, consultar o histórico completo de manutenções realizadas, atualizar a quilometragem atual (odômetro) e acompanhar alertas de troca.
Visitantes	Acessam a plataforma de forma limitada para visualizar informações gerais sobre o funcionamento do sistema, dicas de manutenção preventiva ou busca de oficinas credenciadas sem necessidade de login.

Fonte: Autoria própria (2026)

2.4 Arquitetura física

A arquitetura física adota o modelo cliente-servidor distribuído em infraestrutura de nuvem. A interação com o sistema ocorre por meio de dois tipos de clientes distintos: uma aplicação web dedicada ao uso técnico e administrativo nas oficinas mecânicas e um aplicativo móvel para o acompanhamento dos proprietários de veículos. A execução da lógica de negócios e a persistência dos dados são processadas de forma centralizada em uma plataforma de serviços em nuvem (Backend-as-a-Service), garantindo alta disponibilidade e escalabilidade.

2.4.1 Tecnologias utilizadas

As tecnologias utilizadas no projeto foram selecionadas para garantir a escalabilidade do banco de dados relacional e a agilidade no desenvolvimento multiplataforma, integrando serviços de nuvem modernos:

- **Plataforma de desenvolvimento:** O ambiente de desenvolvimento integrado (IDE) adotado é o **Visual Studio Code**, utilizado por sua versatilidade e suporte nativo aos ecossistemas Flutter e React.
- **Front-end (Oficina):** A interface web voltada para os mecânicos é desenvolvida

em **React**. Esta biblioteca permite a criação de componentes modulares e uma gestão de estado eficiente para formulários complexos de manutenção.

- **Mobile (Cliente):** O aplicativo móvel é construído com o framework **Flutter** (Linguagem Dart). A escolha permite que o cliente tenha uma experiência nativa em Android e iOS a partir de um único código-fonte, facilitando a atualização do odômetro e consulta de histórico.
- **Backend-as-a-Service e Banco de Dados (Supabase):**
 - **PostgreSQL:** Motor de banco de dados relacional que garante a integridade dos dados via chaves estrangeiras e permite a execução de scripts SQL complexos.
 - **Auth:** Gerenciamento nativo de autenticação para controle de acesso de mecânicos e clientes.
 - **PostgREST:** API RESTful gerada automaticamente pelo Supabase, permitindo que as interfaces React e Flutter consumam os dados de forma segura e rápida.
- **Controle de Versão (Git e GitHub):** Para o gerenciamento do código-fonte e controle de versões, utiliza-se o **Git** integrado ao serviço de hospedagem **GitHub**. Esta ferramenta possibilita o rastreamento de alterações, colaboração entre a equipe e o armazenamento seguro do projeto em repositório remoto.

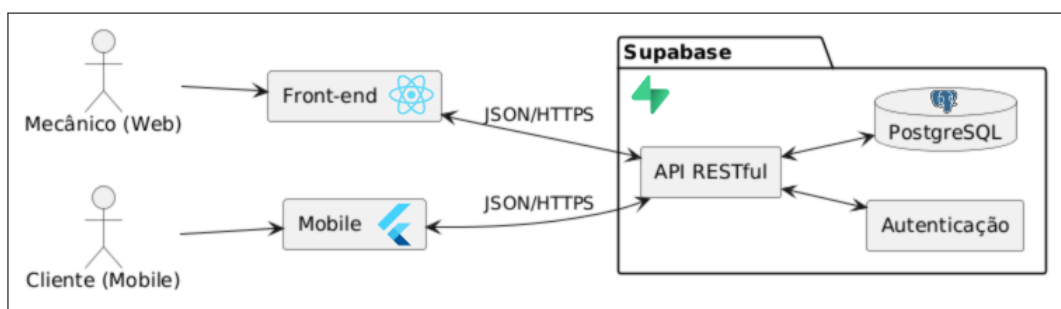
2.5 Arquitetura lógica

Na arquitetura lógica da aplicação, foram definidos os diferentes componentes e módulos que interagem e se organizam para realizar as funcionalidades do Boost+. A estrutura foca na descentralização das interfaces e na centralização da lógica de dados em nuvem. A Figura 1 apresenta de forma simplificada a estrutura utilizada.

- **Front-end (Web):** A camada de interface voltada para o uso nas oficinas (mecânicos) é desenvolvida com *React*. Esta biblioteca permite uma gestão eficiente de estados e a criação de componentes reutilizáveis para o registro detalhado de manutenções e categorias.
- **Mobile (Aplicativo):** A interface destinada aos proprietários de veículos (clientes) é desenvolvida em *Flutter*. Utilizando a linguagem Dart, o aplicativo oferece uma experiência nativa e fluida para a consulta de históricos e atualização de quilometragem.

- *Backend-as-a-Service (BaaS)*: A lógica de back-end e os serviços de infraestrutura são gerenciados pelo *Supabase*. Ele provê os módulos de autenticação de usuários e a camada de serviços que conecta as aplicações cliente ao banco de dados.
- Banco de dados: Os dados da aplicação são armazenados e organizados utilizando o *PostgreSQL*, um sistema de gerenciamento de banco de dados relacional que garante a integridade das informações e o suporte a consultas complexas.
- Tecnologia *web* e integração: A comunicação entre o front-end web, o aplicativo mobile e o *back-end* é feita por meio de uma *API RESTful*, utilizando o padrão JSON para a troca de dados. A *API Restful* é um tipo de interface de programação de aplicações (API) que se baseia em recursos (como URLs) e utiliza os métodos (*GET*, *POST*, *PUT*, *DELETE*, etc.) para realizar operações como leitura, criação, atualização e exclusão de dados.

Figura 1 – Arquitetura lógica com as principais tecnologias.



Fonte: Autoria própria (2026)

2.5.1 Requisitos funcionais

Quadro 2 – Requisitos funcionais

No	Caso de Uso	Descrição resumida
RF-01	Cadastro / Autenticação	O sistema deve permitir que mecânicos e clientes criem contas, façam login e recuperem senhas.
RF-02	Gerenciar Perfil	Usuários podem editar seus dados pessoais (nome, e-mail, telefone) e preferências de notificação.
RF-03	Cadastrar Veículo	O mecânico ou o cliente pode registrar um veículo informando placa, modelo, marca e ano.
RF-04	Registrar Manutenção	O mecânico deve registrar serviços realizados (ex: troca de óleo), incluindo data, quilometragem atual e especificações técnicas (marca, viscosidade).
RF-05	Definir Prazos de Validade	Ao registrar um serviço, o mecânico deve inserir a validade em quilômetros (ex: 10.000km) e em tempo (ex: 6 meses).
RF-06	Consultar Histórico	O cliente deve visualizar todos os registros de manutenção vinculados aos seus veículos de forma cronológica.
RF-07	Atualizar Odômetro	O cliente pode inserir a quilometragem atual do veículo manualmente para que o sistema recalcule a vida útil das peças.
RF-08	Vincular Cliente ao Veículo	O mecânico deve conseguir buscar um cliente (por CPF ou e-mail) para vincular um registro de manutenção ao perfil correto.
RF-09	Painel de Status (Dashboard)	O sistema deve exibir indicadores visuais (cores: verde, amarelo, vermelho) sobre a saúde de cada componente monitorado.
RF-10	Alertas e Notificações	O sistema deve enviar alertas automáticos quando a quilometragem ou a data de uma manutenção estiver próxima do limite.
RF-11	Busca e Filtragem	Permitir que o usuário busque manutenções passadas por tipo de serviço, data ou placa do veículo.
RF-12	Relatórios de Despesas	Gerar um resumo de quanto foi gasto em manutenções em um determinado período.
RF-13	Manter Categorias	Permitir que o mecânico cadastre, edite ou exclua categorias de serviços e peças para organizar os registros de manutenção.

2.5.2 Requisitos de regras de negócio

Os requisitos de regras de negócios são readequados para a aplicação e apresentados no Quadro 3.

Quadro 3 – Requisitos de regras de negócio

ID	Área	Descrição
RGN-01	Cadastro	O cadastro de qualquer usuário (Mecânico ou Cliente) é único e vinculado obrigatoriamente a um CPF válido.
RGN-02	Veículo	Um veículo só pode ser cadastrado no sistema se possuir uma placa válida e única, não sendo permitida a duplicidade de placas na base de dados.
RGN-03	Manutenção	Para registrar uma manutenção, é obrigatório informar a quilometragem atual do veículo, a data do serviço e ao menos um item revisado ou trocado.
RGN-04	Integridade	Um registro de manutenção, após ser finalizado e salvo pelo mecânico, não pode ter sua quilometragem ou data alteradas, garantindo a veracidade do histórico.
RGN-05	Odômetro	A quilometragem atual informada em um novo serviço nunca pode ser inferior à última quilometragem registrada no histórico daquele veículo.
RGN-06	Privacidade	Dados de contato do cliente (telefone e e-mail) não devem ser expostos a outros usuários, exceto ao mecânico responsável pelo atendimento atual.
RGN-07	Cálculo de Validade	O sistema deve calcular a data de "Próxima Troca" baseando-se no que ocorrer primeiro: o limite de quilometragem atingido ou o prazo em meses estipulado pelo mecânico.
RGN-08	Notificações	Alertas de manutenção preventiva devem ser disparados automaticamente 15 dias antes da data de validade ou quando faltarem 500 km para o limite estipulado.
RGN-09	Histórico	Os registros de manutenções passadas servem como registro histórico imutável e não podem ser excluídos, apenas ocultados em caso de erro administrativo devidamente justificado.

Fonte: Autoria própria (2026)

2.5.3 Requisitos não funcionais

Os requisitos não funcionais são listados no Quadro 4.

Quadro 4 – Requisitos não funcionais

ID	Categoria	Descrição
RNF-01	Segurança e Privacidade	O sistema deve estar em conformidade com a LGPD, garantindo que senhas e dados sensíveis (CPF, telefone) sejam armazenados com criptografia.
RNF-02	Usabilidade	A interface deve ser responsiva, adaptando-se perfeitamente a smartphones (uso do cliente) e tablets/desktops (uso da oficina).
RNF-03	Desempenho	O tempo de carregamento do histórico de manutenções deve ser satisfatório, mesmo em conexões de dados móveis (3G/4G).
RNF-04	Acessibilidade	A interface deve garantir contraste adequado e suporte a leitores de tela para usuários com deficiência visual.
RNF-05	Confiabilidade	O sistema deve possuir disponibilidade de 99%, garantindo que o mecânico sempre consiga registrar os dados durante o horário comercial.
RNF-06	Segurança de Rede	Toda a comunicação entre o aplicativo (cliente) e o servidor deve ser criptografada utilizando o protocolo HTTPS.
RNF-07	Compatibilidade	A aplicação deve ser compatível com as versões mais recentes do Android e iOS, além dos navegadores Edge, Chrome, Firefox e Safari.

Fonte: Autoria própria (2026)

2.5.4 Especificações funcionais

Aborda-se agora, o detalhamento das especificações funcionais do sistema na seguinte ordem:

- Especificação UC 001: Registrar Manutenção.
- Especificação UC 002: Cadastrar Veículo.
- Especificação UC 003: Consultar Histórico.
- Especificação UC 004: Atualizar Odômetro.
- Especificação UC 005: Gerar Alerta de Troca.
- Especificação UC 006: Manter categorias.

2.5.4.1 Especificação UC 0001: Registrar Manutenção

Boost+**ESPECIFICAÇÃO FUNCIONAL: UC 0001 - Registrar Manutenção****Descrição Resumida do Caso de Uso**

Este caso de uso descreve as funcionalidades para o mecânico inserir dados técnicos de serviços realizados.

Atores	Mecânico
Pré-Condição	O Mecânico deve estar autenticado no sistema.
Fluxo Principal	P01. O mecânico busca o veículo pela placa ou CPF do cliente. (Interface001). P02. O sistema exibe os dados atuais do veículo. (Interface002). P03. O mecânico seleciona "Registrar Nova Manutenção". P04. O mecânico preenche: Tipo de serviço, KM atual, Marca/Especificação da peça e Validade (KM e Meses). P05. O mecânico clica em "Salvar Manutenção". P06. O sistema valida os dados e exibe MSG001.
Exceções	E01 [P04] - Dados nulos 01. O sistema identifica campos obrigatórios vazios e exibe MSG002. 02. Retorna ao passo [P04]. E02 [P04] - KM inconsistente 01. A quilometragem inserida é menor que a última registrada. O sistema exibe MSG003. 02. Retorna ao passo [P04].
Requisitos	RF-03, RF-12.
Regras de Negócio	RGN-08, RGN-10
Listagem de Mensagens	MSG001: Manutenção registrada com sucesso. MSG002: Preencha os dados obrigatórios. MSG003: A KM atual não pode ser inferior à última registrada.
Interfaces	Interface001: Tela Inicial (Figura ??).

2.5.4.2 Especificação UC 0002: Cadastrar Veículo

Boost+**ESPECIFICAÇÃO FUNCIONAL: UC 0002 - Cadastrar Veículo****Descrição Resumida do Caso de Uso**

Este caso de uso descreve a inclusão de um novo automóvel na base de dados para acompanhamento histórico.

Atores	Mecânico ou Cliente
Pré-Condição	O usuário deve estar autenticado no sistema.
Fluxo Principal	P01. O usuário clica no botão "Adicionar Veículo". P02. O sistema exibe o formulário de cadastro (Interface003). P03. O usuário preenche: Placa, Marca, Modelo e Ano. P04. O usuário clica em "Salvar". P05. O sistema verifica a unicidade da placa e exibe MSG004.
Exceções	E01 [P05] - Veículo já cadastrado 01. O sistema identifica que a placa já existe na base e exibe MSG005. 02. Retorna ao passo [P03].
Pós-Condição	Veículo vinculado ao perfil do usuário.
Produtos Gerados	Registro de veículo no banco de dados.
Requisitos	RF-03.
Regras de Negócio	RGN-02.
Listagem de Mensagens	MSG004: Veículo cadastrado com sucesso. MSG005: Esta placa já está cadastrada no sistema.
Interfaces	Interface003: Tela de Cadastro de Veículo.

2.5.4.3 Especificação UC 0003: Consultar Histórico

Boost+**ESPECIFICAÇÃO FUNCIONAL: UC 0003 - Consultar Histórico****Descrição Resumida do Caso de Uso**

Permite a visualização cronológica de todos os serviços realizados em um veículo específico.

Atores	Cliente
Pré-Condição	O cliente deve possuir ao menos um veículo vinculado.
Fluxo Principal	P01. O cliente seleciona o veículo na tela inicial. P02. O sistema exibe a lista de manutenções (Interface004). P03. O cliente clica em um serviço para ver detalhes. P04. O caso de uso termina.
Fluxo Alternativo	A01 [P02] - Filtrar por categoria 01. O usuário seleciona um filtro (ex: Óleo de motor). 02. O sistema atualiza a lista.
Exceções	E01 [P02] - Inexistência de registros 01. O sistema exibe MSG006.
Requisitos	RF-06, RF-11.
Regras de Negócio	RGN-08, RGN-10.
Listagem de Mensagens	MSG006: Nenhuma manutenção registrada atualmente.
Interfaces	Interface004: Tela de Histórico.

2.5.4.4 Especificação UC 0004: Atualizar Odômetro

Boost+**ESPECIFICAÇÃO FUNCIONAL: UC 0004 - Atualizar Odômetro****Descrição Resumida do Caso de Uso**

Funcionalidade para o cliente manter a quilometragem atualizada e garantir a precisão dos cálculos.

Atores	Cliente
Pré-Condição	O usuário deve estar autenticado e com veículo selecionado.
Fluxo Principal	P01. O cliente acessa o dashboard do veículo. P02. O cliente digita a nova KM e confirma. P03. O sistema atualiza as previsões de troca e exibe MSG007.
Exceções	E01 [P02] - KM inválida 01. KM inserida menor que o registro anterior. Exibe MSG003.
Requisitos	RF-07.
Regras de Negócio	RGN-05, RGN-07.
Listagem de Mensagens	MSG007: Quilometragem atualizada. Previsões recalculadas.

2.5.4.5 Especificação UC 0005: Gerar Alerta de Troca

Boost+**ESPECIFICAÇÃO FUNCIONAL: UC 0005 - Gerar Alerta de Troca****Descrição Resumida do Caso de Uso**

Caso de uso disparado pelo sistema para notificar o usuário sobre manutenções preventivas.

Atores	Sistema
Pré-Condição	KM atual ou Data atingem o limite de segurança.
Fluxo Principal	P01. O sistema verifica prazos periodicamente. P02. Identifica item próximo ao vencimento (RGN-09). P03. Gera notificação push e alerta visual MSG008.
Requisitos	RF-10.
Regras de Negócio	RGN-09.
Listagem de Mensagens	MSG008: Atenção: Troca de [Item] próxima do vencimento.

2.5.4.6 Especificação UC 0006: Manter Categorias

Boost+**ESPECIFICAÇÃO FUNCIONAL: UC 0006 - Manter Categorias****Descrição Resumida do Caso de Uso**

Este caso de uso descreve as funcionalidades para o mecânico cadastrar, editar e excluir as categorias de serviços (ex: Troca de Óleo, Sistema de Freios) dentro do sistema.

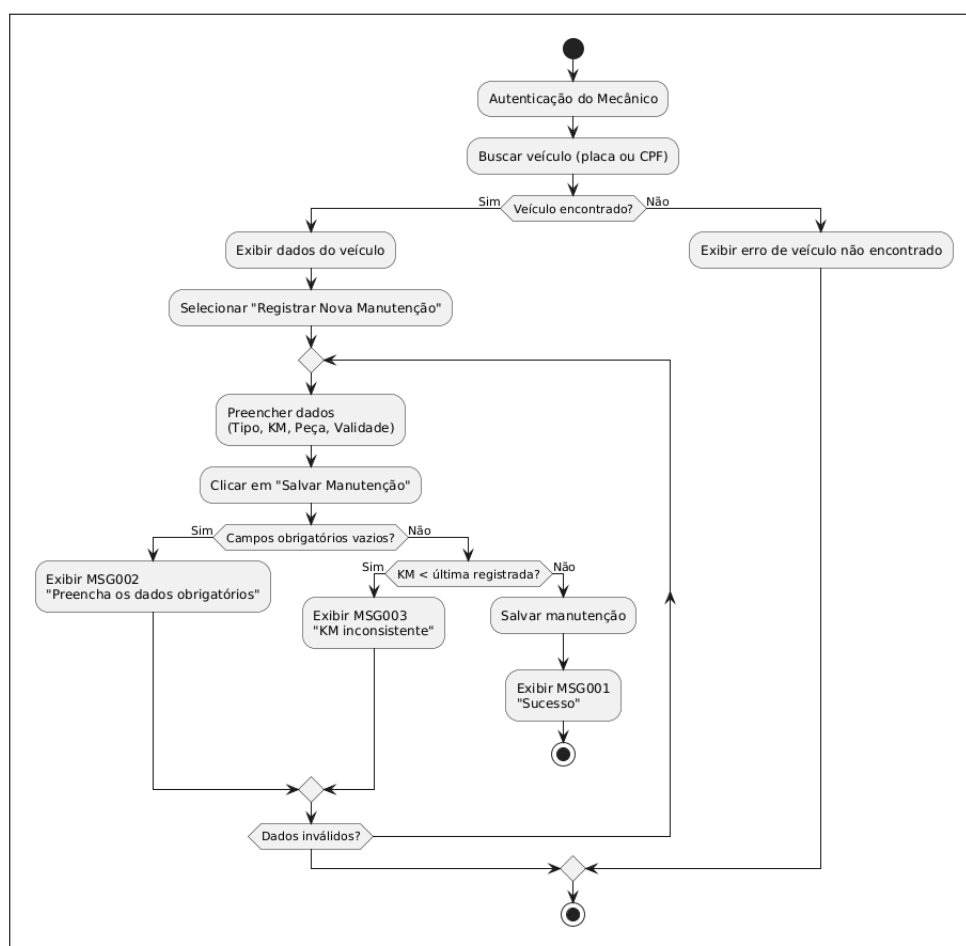
Atores	Mecânico
Pré-Condição	O usuário precisa estar autenticado na plataforma.
Fluxo Principal	P01. O mecânico seleciona a opção "Categorias" no menu de configurações. P02. O sistema exibe a lista de categorias já cadastradas. P03. O mecânico clica no botão "Adicionar Nova Categoria". P04. O sistema exibe formulário com campos de Nome e Descrição da categoria. P05. O mecânico preenche os dados e clica em "Salvar". P06. O sistema valida, salva os dados e exibe MSG009.
Fluxo Alternativo	A01 [P02] - Editar categoria 01. O mecânico seleciona uma categoria e altera o nome ou descrição. 02. O sistema salva e exibe MSG010.
Exceções	E01 [P05] - Categoria Duplicada 01. O sistema verifica que o nome da categoria já existe e exibe MSG011.
Pós-Condição	Categorias atualizadas para uso nos registros de manutenção.
Requisitos	RF-10.
Listagem de Mensagens	MSG009: Categoria cadastrada com sucesso. MSG010: Alterações salvas. MSG011: Já existe uma categoria com este nome.

2.5.5 Diagrama de Atividades

Segundo Sommerville (2011), o diagrama de atividades representa a sequência de ações de um processo. No contexto do Boost+, este diagrama é fundamental para visualizar o fluxo operacional das manutenções e as validações de dados realizadas pelo sistema.

A Figura 2 apresenta o fluxo do caso de uso Registrar Manutenção (UC 0001), iniciado quando o mecânico realiza a busca por um veículo específico na plataforma web. A partir desta ação, o profissional insere as especificações do item e o sistema realiza a validação da quilometragem informada, garantindo que o novo registro seja superior ao último histórico. Caso a validação seja positiva, o sistema salva os dados e atualiza o status de validade das peças; caso contrário, uma mensagem de erro é exibida, solicitando a correção dos dados.

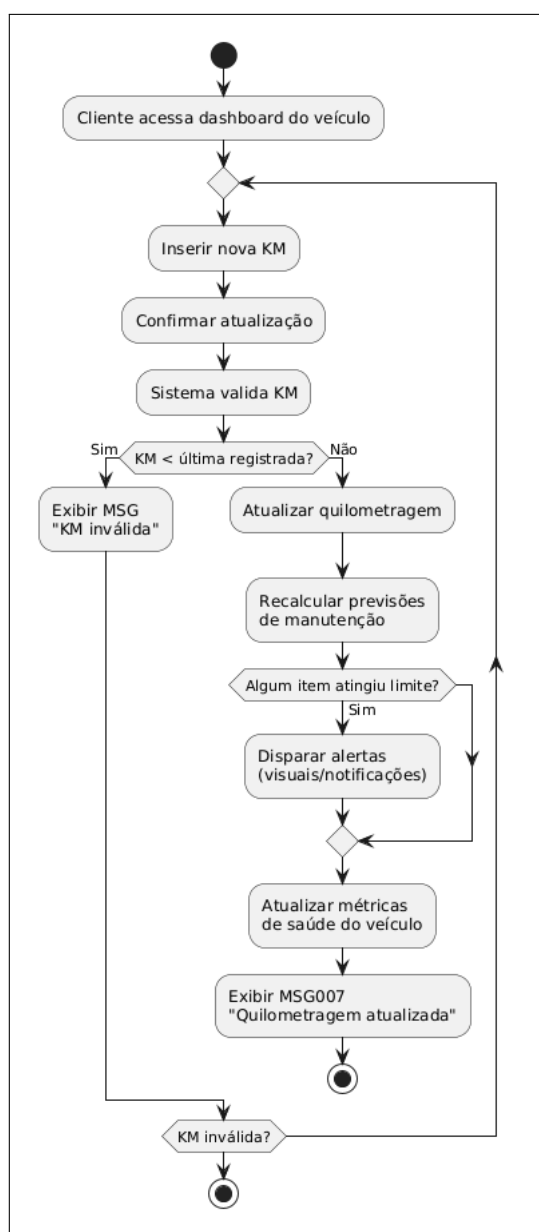
Figura 2 – Diagrama de atividades (Mecânico).



Fonte: Autoria própria (2026)

A Figura 3 detalha a interação do Cliente (UC 0004) ao realizar a Atualização do Odômetro através do aplicativo móvel. O fluxo inicia-se no dashboard do veículo, onde o usuário insere a quilometragem atual visualizada no painel do automóvel. O sistema processa esta informação para recalcular dinamicamente as projeções de validade de todos os componentes monitorados, como óleo e pneus. Caso o limite de algum item seja atingido, o sistema dispara alertas visuais e notificações, encerrando o processo após a atualização das métricas de saúde do veículo.

Figura 3 – Diagrama de atividades (Usuário).



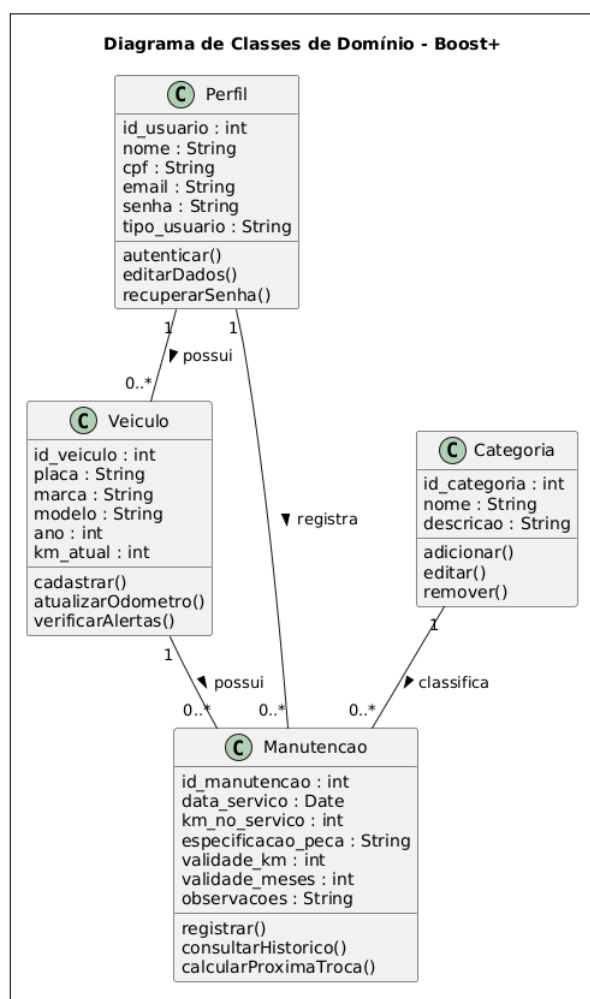
Fonte: Autoria própria (2026)

2.5.6 Diagrama de classes de domínio

Conforme Sommerville (2011), o diagrama de classes mostra as classes de objeto no sistema e as associações entre elas, representando os objetos e suas interações dentro de um domínio específico. Este diagrama fornece a visão estrutural do aplicativo, servindo de base para a implementação do banco de dados relacional e das entidades no código-fonte.

A Figura 4 apresenta o diagrama de classes do sistema, destacando os atributos e os relacionamentos de multiplicidade entre as entidades. O modelo é centrado na classe Veículo, que possui associações com os Perfis de usuários (Mecânicos e Clientes) e uma relação de composição com os registros de Manutenção. Cada registro de manutenção está vinculado a uma Categoria específica, permitindo a organização e filtragem dos dados históricos. Essa estrutura garante que cada intervenção técnica possua rastreabilidade, vinculando o profissional responsável, o veículo atendido e as especificações de validade técnica definidas no momento do serviço.

Figura 4 – Diagrama de Classes de Domínio



Fonte: Autoria própria (2026)

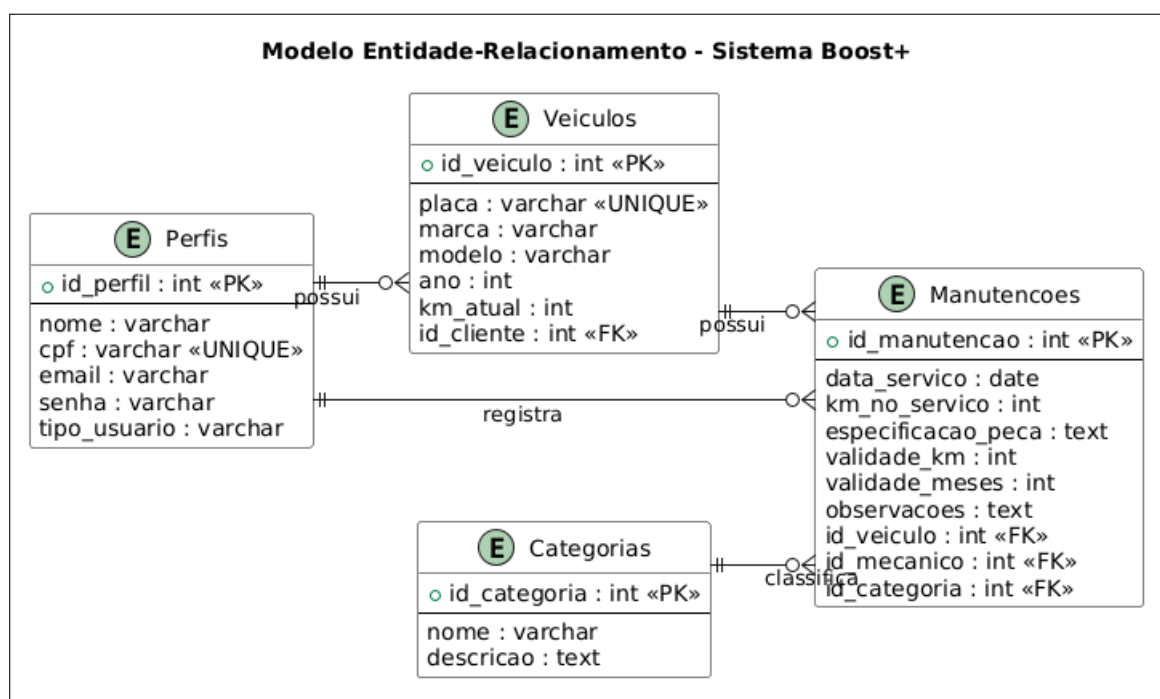
2.5.7 Modelo Entidade Relacionamento (M.E.R)

O Modelo Entidade-Relacionamento (M.E.R) apresenta a estrutura de dados do sistema, evidenciando as entidades, seus atributos e os relacionamentos entre elas. O diagrama reúne elementos fundamentais como Perfis, Veículos, Manutenções e Categorias, demonstrando como os dados se conectam e são organizados na estrutura do banco de dados relacional.

A Figura 5 apresenta o modelo lógico do sistema, onde a integridade dos dados é garantida por meio de chaves estrangeiras (Foreign Keys) que conectam as seguintes entidades:

- **Perfis:** Armazena os dados básicos de usuários, diferenciando mecânicos e clientes por meio de um atributo de tipo. O CPF é utilizado como identificador único para garantir a unicidade de cada cadastro.
- **Veículos:** Representa os automóveis cadastrados, vinculando cada placa a um proprietário específico.
- **Categorias:** Tabela auxiliar que permite a classificação dos serviços (ex: suspensão, motor), conferindo autonomia operacional ao mecânico para organizar os registros de manutenção.
- **Manutenções:** Entidade central que registra o histórico técnico. Ela relaciona-se com o Veículo atendido, o Mecânico responsável pelo serviço e a Categoria correspondente. Esta tabela armazena dados críticos como quilometragem atual, especificações de peças e prazos de validade técnica (KM e tempo). A implementação física deste modelo utiliza o motor PostgreSQL hospedado na plataforma Supabase, assegurando que registros de manutenção passados permaneçam como um histórico imutável, conforme definido nas regras de integridade do sistema.

Figura 5 – Diagrama de Entidade-Relacionamento



Fonte: Autoria própria (2026)

3 Codificação

Descrição do Código

Referências

SOMMERVILLE, I. **Software Engineering**. 9. edition. ed. United States of America: Pearson Prentice Hall, 2011. 22